Exception Handling

Lecture 28

Based on Slides of Dr. Norazah Yusof

Categories of Errors

- There are three categories of errors:
  - syntax errors,
  - runtime errors, and
  - logic errors.
- *Syntax errors* arise because the rules of the language have not been followed. They are detected by the compiler.
- *Runtime errors* occur while the program is running if the environment detects an operation that is impossible to carry out.
- *Logic errors* occur when a program does not perform the way it was intended to.

Exception

- **Exception is a runtime error – an event that occurs during the execution of a program that disrupts the normal flow of instructions.**
- **The program that does not provide code for *catching* and *handling* exceptions will terminate abnormally, and may cause serious problems.**
- **Exceptions occurs for various reasons:**
  - **User enters an invalid input.**
  - **Program attempt to open a file that doesn't exist.**
  - **Program attempt to access an out-of-bounds array element**
  - **and many more…**

Uncaught Exceptions

- **When an exception is thrown, it cannot be ignored.**
- **It must be handled by the program, or by the default exception handler.**
- **When the code in a method throws an exception:**
  - **normal execution of that method stops, and**
  - **the JVM searches for a compatible exception handler inside the method.**

Runtime Errors

```
1
2
3
4
5
6
7
8
9
10
11
12
13
```

```
import java.util.Scanner;

public class ExceptionDemo {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter an integer: ");
    int number = scanner.nextInt();

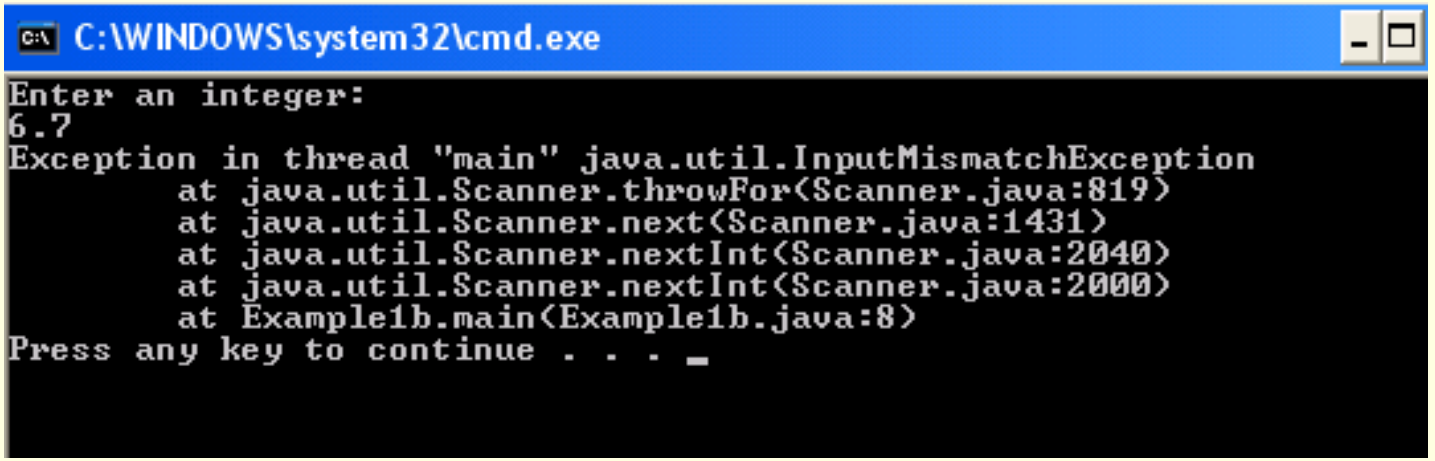
    // Display the result
    System.out.println(
      "The number entered is " + number);
  }
}
```

If an exception occurs on this line, the rest of the lines in the method are skipped and the program is terminated.

Terminated.

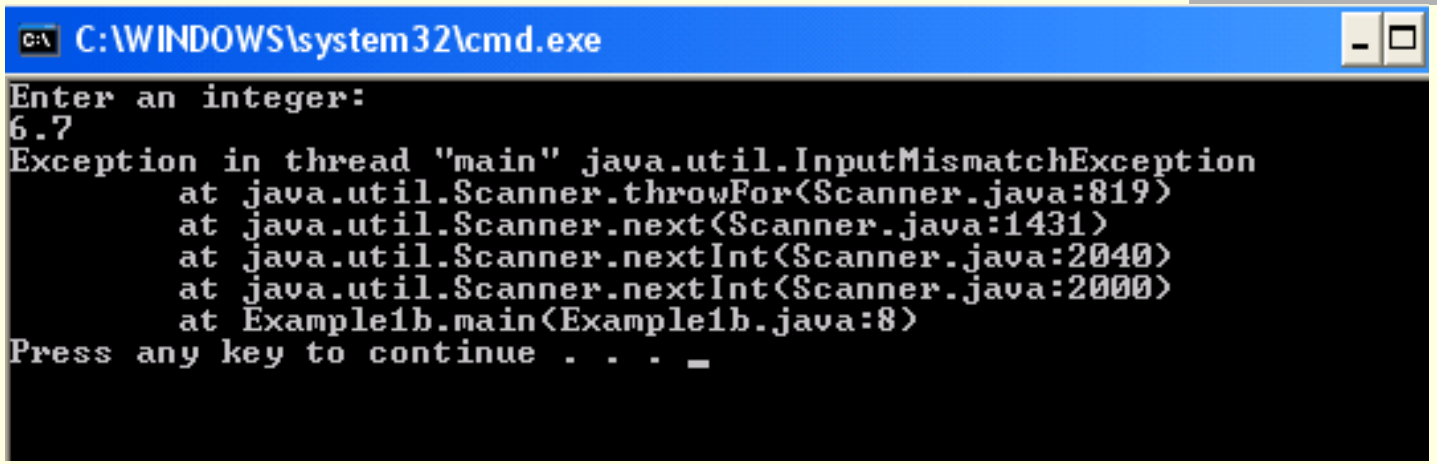
Exception – stack trace

- Several information are displayed in response to the invalid input.



- This information is known as ***stack trace***, that include:
  - The name of the exception,
  - The method call stack

Exception – stack trace



- From the last line of the stack trace:
  - The exception was detected at line 8 of the main method, contains the class name and method (Example1b.main) along with the filename and and line number (Example1b.java:8)
- Moving up the stack:
  - The exception occurred at line 2000 in the nextInt method of the Scanner class.
  - The exception occurred at line 2040 in the overloaded nextInt method of the Scanner class.
  - The exception occurred at line 1431 in the next method of the Scanner class.
  - The exception occurred at line 819 in the throwFor method of the Scanner class.
- The last method in the call chain actually threw an InputMismatchException.

Catch Runtime Errors

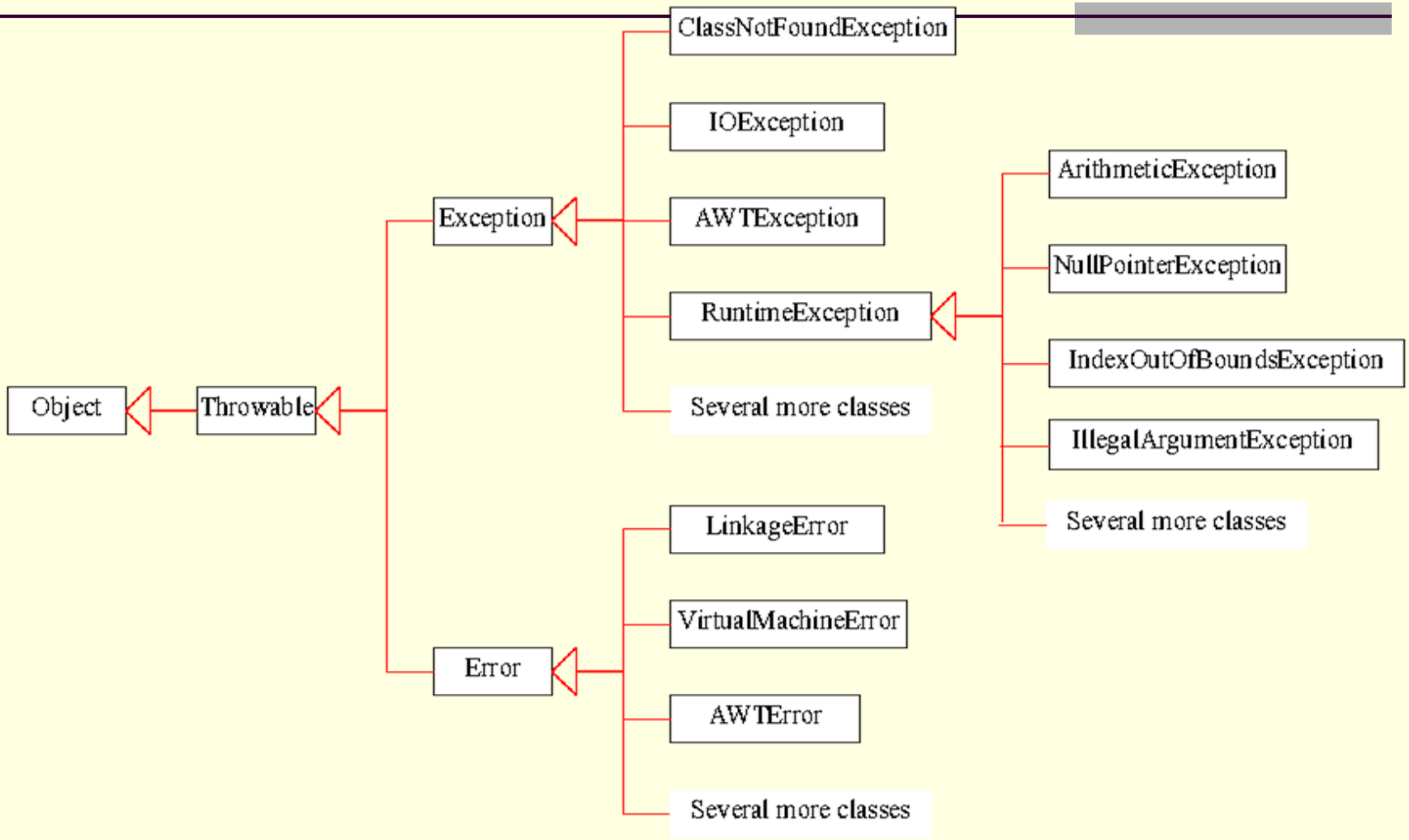
If an exception occurs on this line, the rest of lines in the try block are skipped and the control is transferred to the catch block.

```
 1  import java.util.*;
 2  public class Example1b{
 3    public static void main(String args[]) {
 4       Scanner scanner = new Scanner(System.in);
 5       try {
 6           System.out.println ("Enter an integer:");
 7           int num = scanner.nextInt();
 8           System.out.println ("The number entered is:"+
 9            num);
10     }
11     catch (InputMismatchException ex) {
12       System.out.println ("Cuba Lagi ("+
13            "Kesalahan input: nombor integer diperlukan)");
14       scanner.nextLine(); //discard input
15     }
16    }
   }
```

Exception handling

- **The `try`-block contains the statements that might throw exceptions.**
- **The `catch`-block handles the exceptional case. The catch-block takes a parameter of type `Exception`.**
  - **When a statement in the `try`-block throws an exception, the execution in the `try`-block ends and control passes to the `catch`-block(s).**

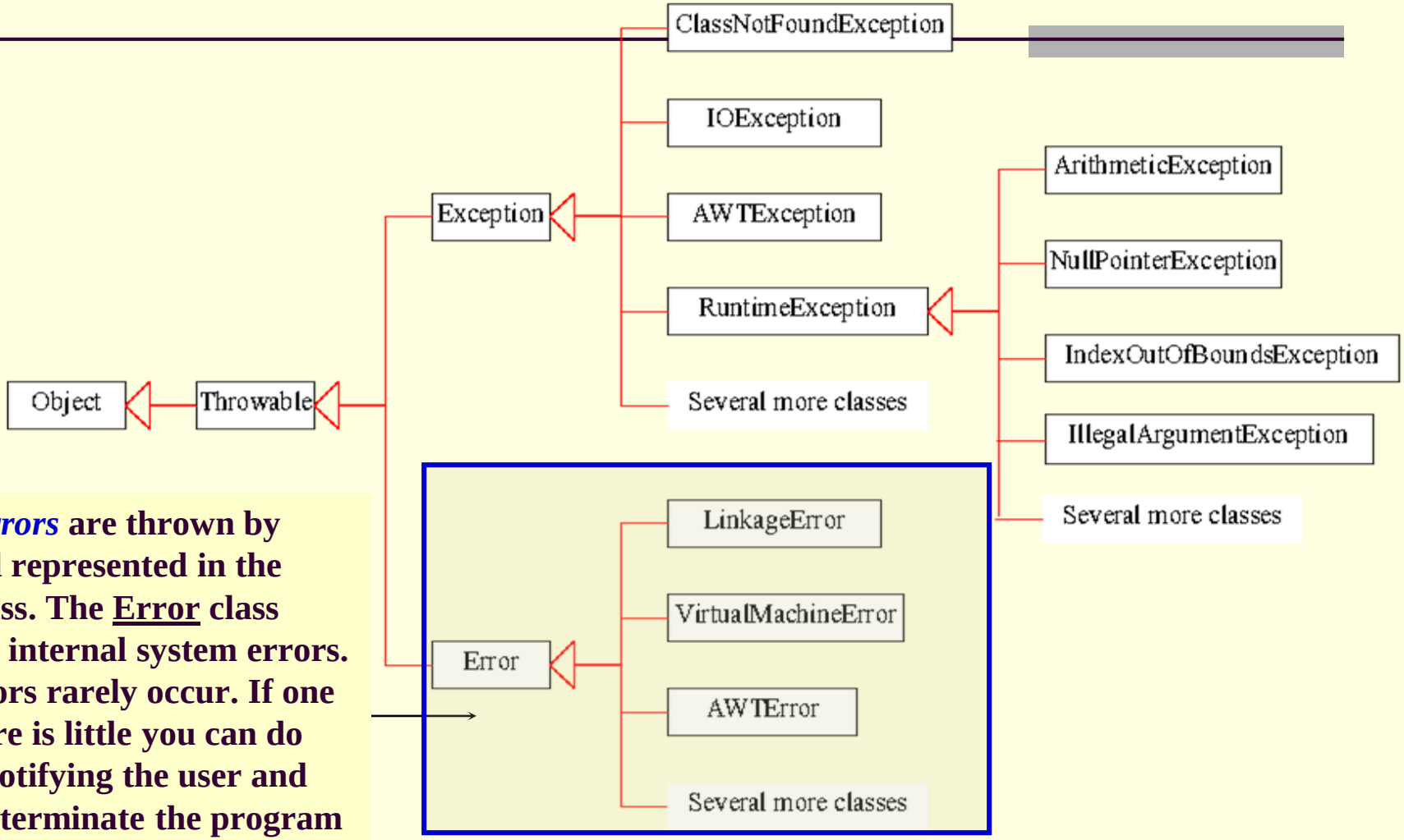
Exception Classes



Exception Classes

- **The exception classes can be classified into three major types:**
  1. **System errors**
  2. **Exceptions**
  3. **Runtime exceptions**

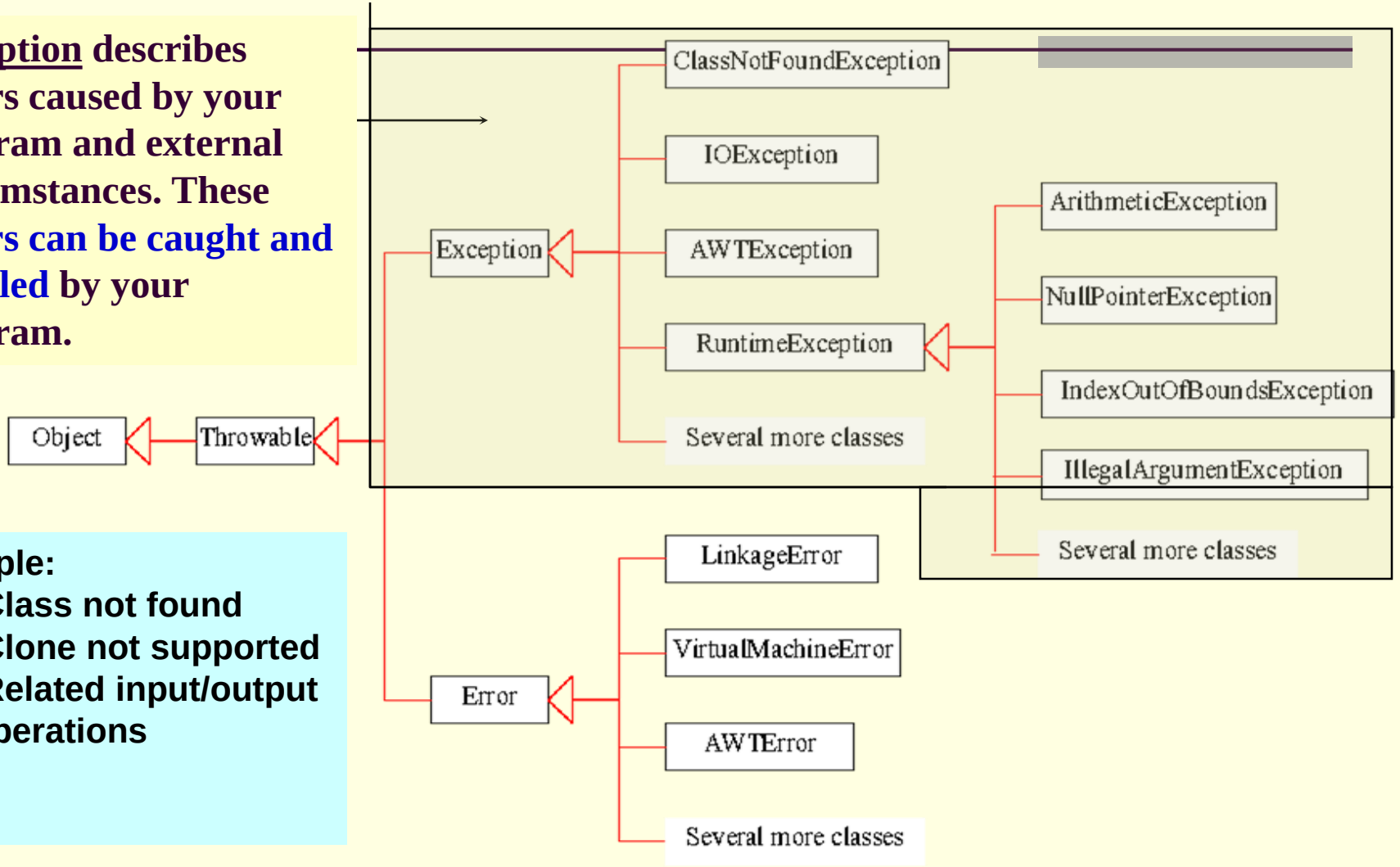
System Errors



System errors **are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.**

Exceptions

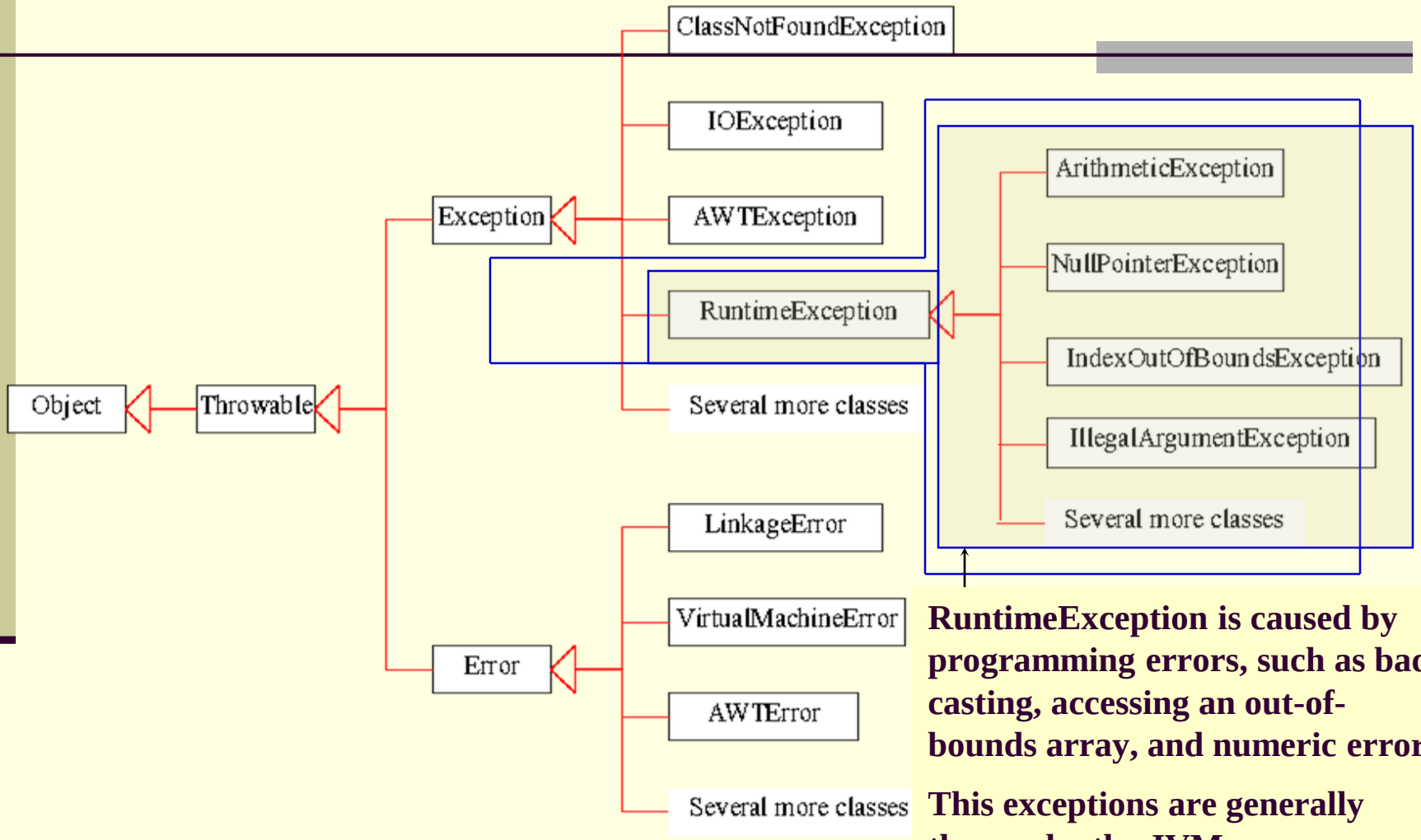
Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



Example:

1. **Class not found**
2. **Clone not supported**
3. **Related input/output operations**

Runtime Exceptions



RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

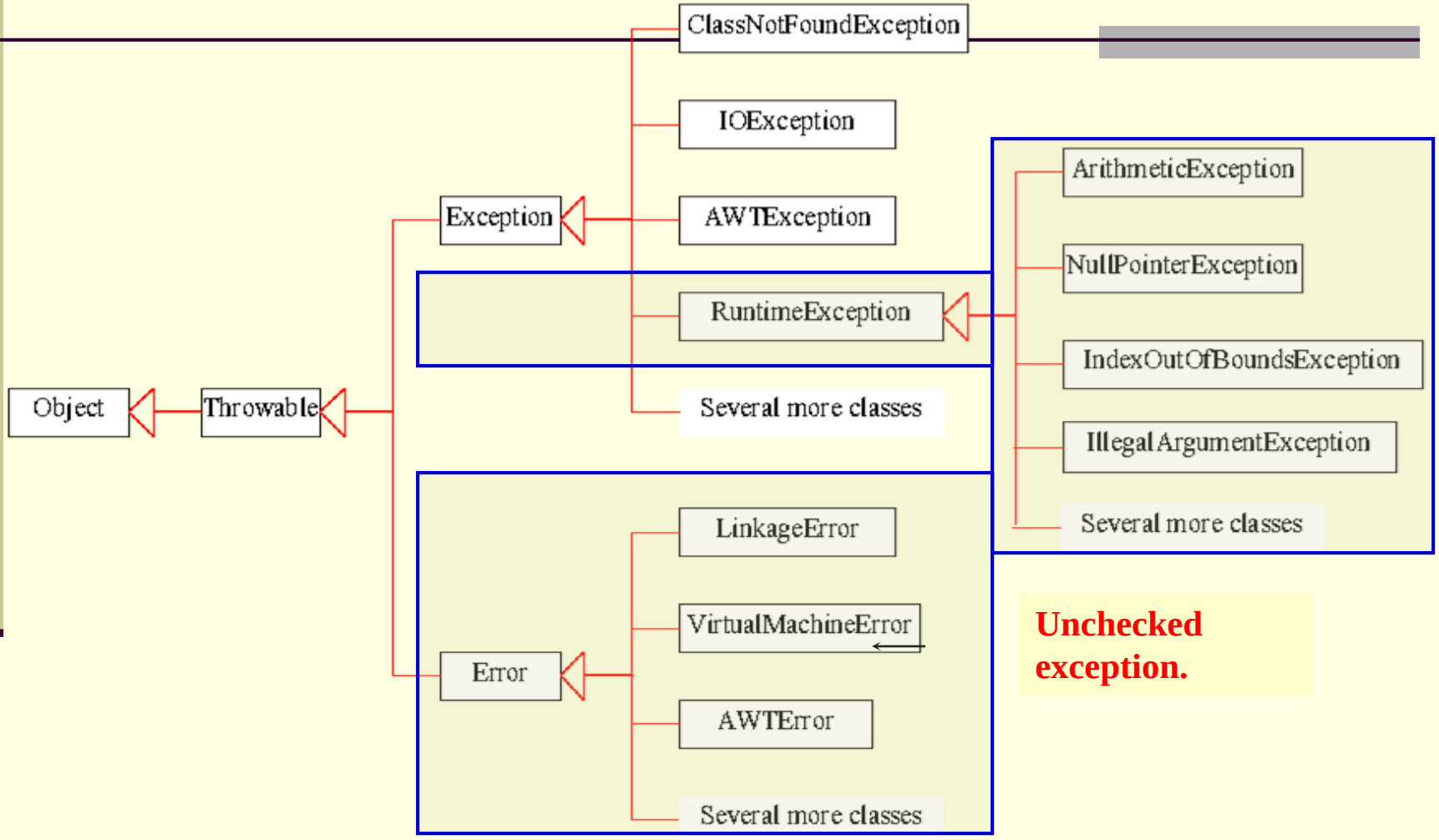
This exceptions are generally thrown by the JVM.

They are known as unchecked exceptions.

Checked and Unchecked Exceptions

- There are two categories of exceptions:
  - unchecked
  - checked
- *Unchecked exceptions* are those that are derived from the `Error` class or the `RuntimeException` class.
- Exceptions derived from `Error` are thrown when a critical error occurs, and should not be handled.
- `RuntimeException` serves as a superclass for exceptions that result from programming errors.

Unchecked Exceptions



Unchecked Exceptions

- **These exceptions can be avoided with properly written code.**
- **With an unchecked exception, however, compiler doesn't force client programmers either to catch the exception or declare it in a throws clause.**
- **All exceptions that are *not* derived from `Error` or `RuntimeException` are *checked exceptions*.**

Unchecked Exceptions

- Is any exception belonging to a subclass of `RuntimeException`.
- It is not checked by the compiler and can occur anywhere in the program.
- In most cases, unchecked exceptions reflect programming logic errors that are not recoverable.
- For example,
  - a NullPointerException is thrown if you access an object through a reference variable before an object is assigned to it;
  - an ArrayIndexOutOfBoundsException is thrown if you access an element in an array outside the bounds of the array.

Checked Exceptions

- **Is one that can be analyzed by the compiler (JVM).**
- **A checked exception is any subclass of Exception, excluding class `RuntimeException` and its subclasses.**
- **You should compulsorily handle the checked exceptions in your code, otherwise your code will not be compiled. i.e you should put the code which may cause checked exception in try block. "checked" means they will be checked at compile time itself.**

Checked Exceptions

- There are two ways to handle checked exceptions. You may declare the exception using a **throws** clause or you may use the **try..catch** block.
- The most perfect example of Checked Exceptions is `IOException` which should be handled in your code compulsorily or else your code will throw a Compilation Error.

Catch or Declare Checked Exceptions

Java forces you to deal with checked exceptions. If a method declares a checked exception (i.e., other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method.

For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

Checked Exceptions

```
// This method will not compile!
public static void main(String args[]){
    BufferedReader input = new BufferedReader
            (new InputStreamReader(System.in));
    String inputData;
    int num;
    System.out.println ("Enter an integer:");
    inputData = input.readLine();
    num = Integer.parseInt(inputData);
    System.out.println ("The square is:" +
  (num*num));
  }
```

Checked Exceptions

- **The code in this method is capable of throwing checked exceptions.**
- **Ways to solve:**
- **Either:**
  - **The keyword `throws` can be written at the end of the method header, followed by a list of the types of exceptions that the method can throw.**

```
public static void main(String args[]) throws
  IOException {
```

```
//program 11.2b
import java.io.*;
public class Example2 {
  public static void main(String args[]) throws
  IOException  {
    BufferedReader input = new BufferedReader (new
  InputStreamReader(System.in));
    String inputData;
    int num;
  System.out.println ("Enter an integer:");
    inputData = input.readLine();
    num = Integer.parseInt(inputData);
    System.out.println ("The square is:"+(num*num));
  }
}
```

- **OR:**

```
//program 11.2c
import java.io.*;
public class Example3 {
  public static void main(String args[]) {
   try {
     BufferedReader input = new BufferedReader (new
   InputStreamReader(System.in));
     String inputData;
     int num;
       System.out.println ("Enter an integer:");
     inputData = input.readLine();
     num = Integer.parseInt(inputData);
     System.out.println ("The square is:"+(num*num));
      }
      catch (NumberFormatException ex) {
    System.out.println ("Wrong data type");
      }
      catch (IOException ex){
     System.out.println ("input problem");
      }
  }
}
```

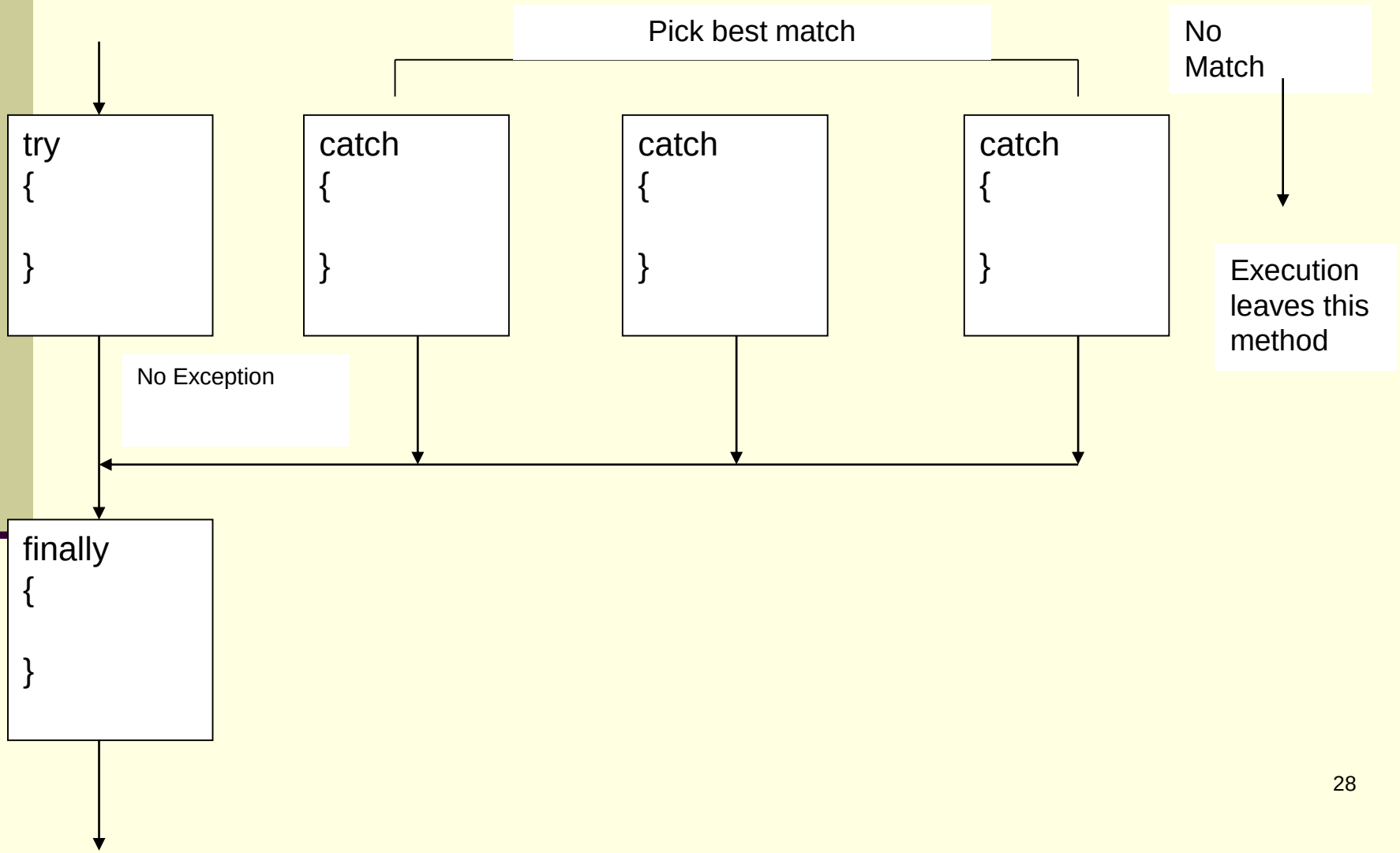
Exception Handler in the method where it occurs

```
try
{
   //the code which will cause the exception
}
 catch(SomeExceptionType ex)
{
   //the code which will handle an object of that
   //exception class
}
:://more catch blocks
 catch(AnotherExceptionType ex)
{    }
 finally
{//the code which will always executed}
//Statements following the structure
```

Exception Handler in the method where it occurs

- **The normal case is handled in a `try`-block.**
- **The exceptional case is handled in a `catch`-block.**
- **The `catch`-block takes a parameter of type `Exception`.**
- **If an exception is *thrown*, execution in the `try`-block ends and control passes to the `catch`-block(s) after the `try`-block**

How try, catch{} & finally{} works



Self test 1

```
1   import java.io.*;
2   public class TestException {
3      public static void main (String[] args) throws
4      IOException {
5       try {
6         method();
7         System.out.println("After method call");
8       }
9       catch (ArithmeticException ex ){
10        System.out.println("ArithmeticException");
11      }
12      catch (RuntimeException ex ){
13        System.out.println("RunTimeException");
14      }
15      catch (Exception ex ){
16        System.out.println("Exception");
17      }
18     }
19  static void method() throws Exception {
20     System.out.println(1/0);
        }
    }
```

Self test 1

```
ArithmeticException
```

```
1   import java.io.*;
2   public class TestException {
3      public static void main (String[] args) throws IOException {
4       try {
5         method();
6         System.out.println("After method call");
7       }
8       catch (ArithmeticException ex ){
9         System.out.println("ArithmeticException");
10      }
11      catch (RuntimeException ex ){
12        System.out.println("RunTimeException");
13      }
14      catch (Exception ex ){
15        System.out.println("Exception");
16      }
17     }
18  static void method() throws Exception {
19   try{    String s="abc";
20           System.out.println(s.charAt(3));
21   }
22   catch (RuntimeException ex ){
23    System.out.println("RunTimeException in method()");
24   }
25   catch (Exception ex ){
26    System.out.println("Exception in method()");
27   }
28  }
```

Self test 2

```
RunTimeException in method()
After method call
```

Rules

- **A program CANNOT have more than one `catch` clause per exception type in the same `try` statement.**
- **If you are handling multiple exceptions in the same `try` statement, then you MUST handle the more specialized exception classes before the more general exception classes.**
- **The finally block is an option block that can appear after all the catch blocks.**

Exercises

- **Self tests**
  - **Question 1- 6, page 210**
- **Lab Exercises:**
  - **Question 7, page 211**
  - **Question 8, page 212**
  - **Question 9, page 213**
  - **Question 10, page 213**